

COMPILER DESIGN LABORATORY

PHASE 1 REPORT - PROBLEM DEFINITION & SYSTEM DESIGN

Type Check

A Type-Checked Expression Language - Design & Prototype

Student Name	Dhruv Ramdohkar
Registration No.	24BKT0042
Course	BCSE307P - Compiler Design Laboratory
Submission	Phase 1 - Individual Project

1. Abstract

This project designs and implements “Type Check” - a small, statically-typed expression language, along with a compiler front-end that performs lexical analysis, syntax analysis, and semantic type checking.

Unlike simple calculators that evaluate expressions blindly, this system validates the types of variables and operands before execution, catching errors such as adding an integer to a string at compile time rather than at runtime.

The front-end is implemented using Flex for tokenization and a hand-written recursive-descent parser in C++ that builds an Abstract Syntax Tree (AST), backed by a symbol table for scope and type tracking.

2. Introduction and Motivation

Static type checking is one of the most important techniques used in modern compilers and language processors. Languages such as TypeScript, Java, and Rust catch a large class of bugs before a program ever runs, simply by verifying that operations are performed on compatible types.

2.1 Why This Project

- **Catch Bugs Earlier** - static type checking finds errors at compile time instead of during execution.
- **Understand Real Compilers** - implementing type checking bridges lexical/syntax analysis with true semantic analysis, the heart of a compiler front-end.
- **Practical Lex + C++ Skills** - directly builds on Flex and C++ concepts, extended into a complete mini language-processing pipeline.

3. Problem Statement

Most beginner-level expression evaluators parse and execute expressions directly, without verifying that the operations being performed are type-valid. This allows type errors to surface only at runtime - often as crashes or silently incorrect results - instead of being caught early during compilation.

There is a need for a small, self-contained language-processing system that can tokenize source code, parse it into a structured representation, and statically verify type correctness, mirroring how production compilers catch bugs before a program ever runs.

WITHOUT TYPE CHECKING

```
int x = 5;
string y = "hi";
print(x + y);    // crashes at
runtime
```

WITH TYPE CHECKING

```
int x = 5;
string y = "hi";
print(x + y);
```

```
// Error (line 3): cannot apply "+" to int and string
```

4. Objectives

- Design a grammar and token set for an expression language with int, float, string and bool types.
- Implement a lexical analyzer using Flex to recognize keywords, literals, identifiers and operators.
- Build a recursive-descent parser in C++ that constructs an Abstract Syntax Tree (AST).
- Implement a symbol table for variable declaration, scope and type tracking.
- Implement a semantic analyzer that performs static type checking over the AST.
- Report meaningful, line-numbered errors for lexical, syntax and type mismatches.

5. Scope of the Project

5.1 In Scope (Phase 1 – Phase 3)

- Lexical analysis for keywords, literals, identifiers and operators.
- Recursive-descent parsing into an AST.
- Symbol table with scope and type tracking.
- Static type checking with error reporting.
- Support for int, float, string and bool types.

5.2 Out of Scope

- Full intermediate code generation is a stretch goal for Phase 3.
- Code optimization passes.
- Target / machine code generation.
- User-defined functions and complex control flow.
- GUI / IDE integration.

6. Background Study

6.1 Lex / Flex

Studied regular-expression based lexical analyzer generators; Flex converts token patterns into a fast DFA-based scanner in C.

6.2 Recursive-Descent Parsing

Reviewed top-down parsing where each grammar rule maps to a function - simple to hand-write and to trace during debugging.

6.3 Static Type Systems

Examined static type-checking rules used in C and Java (type compatibility, implicit widening of int→float, mismatched-type errors) as a reference model for Type Check.

6.4 Symbol Tables

Studied data structures (hash maps) used by compilers to store identifier names, types, and scope information.

7. Compiler Design Concepts Involved

- Lexical Analysis
- Syntax Analysis (Parsing)
- Abstract Syntax Tree Construction
- Symbol Table Management
- Semantic Analysis
- Static Type Checking
- Error Detection and Reporting
- Scope Management

8. Language Specification and Grammar

8.1 Token Categories

Token Category	Pattern / Examples
Keywords	int, float, string, bool, print
Identifiers	[a-zA-Z_][a-zA-Z0-9_]*
Int / Float Literal	42 3.14
String Literal	"hello"
Operators	+ - * / = == != < >
Punctuation	; () { }

8.2 Grammar (Simplified BNF)

```

program          -> stmt_list stmt          ->
decl_stmt | assign_stmt | print_stmt decl_stmt          ->
TYPE ID ('=' expr)? ';' assign_stmt          -> ID '=' expr ';'
print_stmt       -> 'print' '(' expr ')' ';' expr
-> term (('+'|'-') term)* term          -> factor
(('*'|'/') factor)* factor          -> ID | INT | FLOAT |
STRING | '(' expr ')'

```

9. Proposed Methodology

- **Grammar Design** - finalize tokens, keywords and BNF grammar for the expression language.
- **Lexer (Flex)** - implement a .l file with regex rules mapping source text to tokens.
- **Parser (C++)** - hand-write a recursive-descent parser that builds an AST from tokens.
- **Symbol Table** - track declared identifiers, their types and scope during parsing.
- **Type Checker** - traverse the AST, verify operand types, flag mismatches with errors.
- **Testing** - run valid, invalid and edge-case programs; verify error messages.

10. System Architecture

Source code is tokenized by the Flex-generated lexer, parsed into an AST by a recursive-descent parser, and the semantic analyzer walks the AST, consulting the symbol table to statically verify types before producing output or an error report.

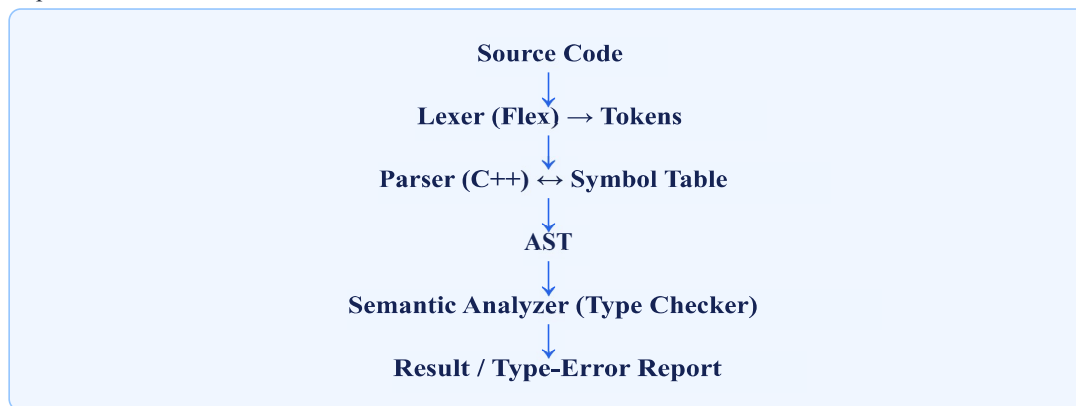


Figure 1: End-to-end pipeline of the Type Check compiler front-end.

11. Technology Stack

Tool / Technology	Purpose
Flex	Lexical analyzer generator - converts regex token rules into a C scanner.
C++	Parser, AST, symbol table and semantic (type-checking) logic.
GCC / G++	Compilation and execution-lexer and parser are built and run directly via terminal commands (flex + g++), as practiced in lab.
VS Code	Development environment for editing and debugging.
Git	Version control and development history tracking.

12. Initial Prototype

A working Flex scanner (tokenizer.l) classifies raw Type Check source into tokens - the first stage of the pipeline shown earlier.

TOKENIZER.L

```
%{
#include "tokens.h"
}%

%%

"int"|"float"|"string"|"bool" { return KEYWORD; }

"print"                        { return PRINT; }
[0-9]+\.[0-9]+                 { return FLOAT_LIT; }
[0-9]+                         { return INT_LIT; }
\"^[^\"]*\"                     { return STRING_LIT; }
[a-zA-Z_][a-zA-Z0-9_]*         { return IDENTIFIER; }
"+"|"-"|"*"|"/"|"="           { return OPERATOR; }
";"                            { return SEMI; }
[ \t\n]+                       ; /* skip whitespace */
.                               { return UNKNOWN; }
%%
```

SAMPLE INPUT

```
int x = 10;
float y = 2.5;
print(x + y);
```

TOKEN STREAM (PROTOTYPE OUTPUT)

```
KEYWORD(int) IDENTIFIER(x) OPERATOR(=) INT_LIT(10) SEMI
KEYWORD(float) IDENTIFIER(y) OPERATOR(=) FLOAT_LIT(2.5) SEMI
PRINT OPERATOR(()) IDENTIFIER(x) OPERATOR(+) IDENTIFIER(y) OPERATOR(())
SEMI
```

13. Conclusion and Phase 2 Preview

Phase 1 establishes a clear problem definition, a designed grammar, and a working lexer prototype. This foundation directly carries into Phase 2, where the parser, symbol table, and type checker will be fully implemented and integrated.

- Complete the recursive-descent parser producing a full AST.

- Implement the symbol table with insert / lookup / scope handling.
- Implement type-checking rules across all supported operators.
- Build a test suite of valid, invalid and edge-case programs.

14. References

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. Compilers: Principles, Techniques, and Tools (2nd ed.).
- [2] Flex - The Fast Lexical Analyzer, official documentation, <https://westes.github.io/flex/manual/>
- [3] TypeScript Handbook - Type Checking, <https://www.typescriptlang.org/docs/handbook/>